

基于 LSTM 的网络热点事件传播速度预测模型研究

摘要

在移动互联网与社交媒体深度融合的背景下，微博、抖音等平台已成为热点事件传播的关键场域。此类事件具有突发性强、演变复杂的特点，其传播速度直接影响舆情走向与干预效果。当前研究多集中于静态热度分析，对传播速度这一动态指标的预测仍面临传统时序模型对非线性特征捕捉不足的挑战。

本文提出一种基于长短期记忆网络（LSTM）的网络热点事件传播速度预测方法。通过构建双层 LSTM 网络架构，引入 Dropout 正则化与早停机制，并采用 Adam 优化器进行训练，增强了模型对时序数据中复杂模式与长期依赖关系的建模能力。实验证明，该方法在传播速度预测任务中表现优于传统时序模型，能更有效识别传播过程中的突变特征。

在理论层面，本研究突破了传统舆情分析聚焦静态热度的范式，实现了向动态传播过程建模的范式转型。在实践层面，所构建模型可为网络综合治理提供量化工具：通过精准预测传播拐点，助力监管部门实现从被动应对到主动预警的网络治理体系，支撑跨部门协同响应与分级分类治理，从而为“健全网络综合治理体系、营造清朗网络空间”的国家战略提供可落地的技术支撑。未来工作将探索多源信息融合及模型在更加复杂的舆情场景中的适应性研究。

关键词：LSTM；网络热点事件；传播速度预测；时间序列；舆情分析

目录

表格与插图清单.....	3
一、绪论	4
(一) 研究背景与选题意义	4
(二) 国内外研究现状.....	5
(三) 创新点	6
二、研究内容与方法	7
(一) 研究思路.....	7
(二) 研究方法.....	8
三、指标体系构建与数据处理.....	9
(一) 指标体系构建	9
(二) 数据采集.....	11
(三) 数据预处理.....	13
四、基于 LSTM 的传播速度预测模型构建	15
(一) 预测目标与样本构造.....	15
(二) LSTM 网络结构设计	16
(三) 训练策略与优化.....	17
(四) 模型评估指标	18
五、实证数据结果检验与模型对比	19
六、研究结论	21
七、现实意义与价值	22
八、政策建议	23
九、研究局限与未来展望	24
(一) 研究局限.....	24

（二）未来展望.....	24
参考文献	25
附录.....	27

表格与插图清单

表 1 指标体系

表 2 部分采集数据展示（5 条）

表 3 数据预处理后部分数据展示（5 条）

图 1 LSTM 网络热点事件传播速度预测模型框架

图 2 前 29 个事件每 4 小时传播速度时序图

图 3 后 29 个事件每 4 小时传播速度时序图

图 4 预测值与真实值的对比曲线

图 5 训练损失与验证损失随训练轮次（Epoch）的变化曲线

图 6 网络热点事件传播速度预测模型对比

一、绪论

（一）研究背景与选题意义

移动网络与社交媒介高度渗透的当下，微博、抖音、知乎、小红书一类平台，早已成为热点事件发酵与扩散的主要场所，这类事件的传播往往具备突然爆发、演变路径曲折、波及面广泛等特性，其扩散的快慢几乎直接塑造了舆论形态、话题讨论热度以及可能带来的社会效应，可是，现有的网络舆情监控体系大多将注意力集中于热度本身的高低，对于“传播有多快”这样一个动态属性的描述和预估，仍显不足。一方面，像 ARIMA 这样传统的时序统计方法，很难有效刻画传播过程中常见的非直线变化、剧烈起伏以及长程关联特征；另一方面，大量研究只做热度总体走向的粗略分析，却未对“速率变化细节”展开精确建模。

与此同时，公共管理与舆情调控的现实需求愈发迫切。当出现突发事件或关注焦点时，管理方迫切需要事先了解扩散的高峰时刻、加速与减缓的阶段以及整个过程可能维持多久，以便从被动的应急处理转变为积极的提前部署，作为一种擅长处理时间序列的深度学习架构，长短期记忆网络（LSTM）借助其门控机制，能较好地抓取序列数据里的远期依赖，这为解决热点事件传播速度的精准估算提供了可行的办法。正因如此，开发一个基于 LSTM 的传播速率预估模型，对于增强舆情研判的科学性、时效性均有着不可忽视的现实作用。

在理论层面：

1. 把 LSTM 在网络舆情领域中的应用进一步推向深入，研究的着眼点由常见的“热度推断”迁移至“传播快慢的预估”，这无疑拓宽了舆情动态特征量化的考察侧面。

2. 建立起一套“热度指数—传播速率—时序推算”的完整分析脉络，为量理解网络热点扩散规律提供了另一种思考架构。

3. 借助与传统时序手段、一般机器学习方法的横向比较，能凸显出 LSTM 在处理传播速率非线性变动方面的长处，从而为后续相关探索给出方法层面的参照。

同时在实践层面：

1. 能为舆情监管和公共事务决策带来帮助：该模型可以提前对热点扩散的节律与关键时间点做出估计，辅助有关机构拟定更具针对性的舆论引导方案。
2. 改善预警环节的准确程度：通过对速率突变信号的早期捕捉，可对潜在的高危事件进行先期处置，减少局面失去掌控的可能。
3. 为平台方的内容管理工作提供参考依据：有助于平台把握热点话题的蔓延规律，继而优化内容审查及流量分配策略，使网络秩序更趋平稳。

（二）国内外研究现状

随着社交媒体的普及和信息传播技术的飞速发展，网络热点事件的传播速度和影响力引起了广泛关注。学者在网络热点事件传播速度预测方面进行了大量研究，主要采用基于统计学、机器学习和深度学习等方法。早期的研究多集中于使用传统的统计学模型，例如自回归积分滑动平均模型（ARIMA）等，这些模型为事件传播趋势的预测提供了一定的基础框架。

随着机器学习技术的兴起，支持向量机（SVM）、随机森林等算法被应用于网络热点事件传播速度预测领域。在一定程度上提高了网络热点事件预测的准确性。

近年来，深度学习模型尤其是长短期记忆网络（LSTM）因其对时间序列数据的优异处理能力，在网络热点事件传播速度预测中得到了广泛应用。国内外学者通过构建基于 LSTM 的预测模型，能够更好地捕捉传播速度随时间变化的复杂模式。例如，一些研究利用 LSTM 模型对社交媒体平台上特定话题的传播速度进行预测，通过分析历史传播数据中的时间依赖关系，取得了较为准确的预测结果（Zheng, 2021; Cheng, 2025）。此外，还有研究将 LSTM 与其他模型进行融合，例如，李若晨与肖人彬（2024）提出了一种基于改进狼群算法（IWPA）优化长短期记忆（LSTM）神经网络的舆情演化预测模型。采用 Halton Sequence 进行初始化，提高种群多样性；设计步长因子进行高斯正弦扰动变换，提高狼群探索开发能力并引入记忆力机制，使用双向记忆种群增加狼群协同合作能力，将改进后的狼群算法应用到 LSTM 神经网络的超参数预测。马永军与朗威（2023）采用擅长处理时间序列数据的 LSTM 模型，引入注意力机制，有效分配网络舆情

各个影响因素的权重,更快地拟合舆情的发展趋势,使得模型预测的速度和精度得到提高,同时也让模型具有一定的可解释性。

国内外研究注重结合实际的场景和需求。考虑到网络热点事件在不同领域的传播特点,如政治、经济、文化等,部分研究针对特定领域的事件(如公共健康事件、自然灾害)进行传播速度预测模型的构建和优化(Zhao 等, 2022; Pan 等, 2025)。此外,还有一些研究关注多源数据与多特征融合,他们将社交媒体数据、情感特征、用户影响力特征等进行整合,为模型提供更全面的信息,从而提高预测的可靠性和准确性。

目前,国内外在基于 LSTM 的网络热点事件传播速度预测模型研究方面取得了一定进展,但仍存在可解释性、适应性等方面的不足,需要进一步深入研究并完善模型算法。

(三) 创新点

1. 关注焦点的转变。本研究突破了传统舆情分析中以静态热度值为核心指标的范式,而是将重点移到传播速率这一动态维度上,这样做能更细致地反映热点从酝酿到爆发的节律与扩散效能,直接回应了早期预警工作的需要,使得研究视角更为贴近实际且具有针对性。

2. 度量指标体系的革新。我们利用熵权法综合考量转发数、评论数、点赞数等多个层面,形成加权后的热度指数,借此克服依赖单一数据源的局限性,再以此为基础推算出速度时序值,给模型供应了信息含量更丰富的输入变量,有利于预估精度的提升。

3. 轻量化推理。精简 LSTM 模型结构,推理阶段只涉及单轮前向传播,单步预测延迟低,能够在毫秒级内完成,满足实时监测对响应速度的苛刻要求。

4. 并行计算支持。模型的前向计算可天然地利用 GPU 张量并行特性对批量样本进行并行处理,显著提升训练与在线推理的吞吐量;同时,在多事件、多时间窗口下,模型副本可进一步部署于多 GPU 或分布式节点,实现大规模并行预警,从而拓展了方法的工程应用边界,兼顾了学术探索与实际工程的双重价值。

二、研究内容与方法

（一）研究思路

第一步，基于转、评、赞等指标，利用熵权法构建综合热度评分，进而计算其单位时间内的变化量，形成时序传播速率序列。第二步，获取公开舆情数据集或实时爬取热点榜单，清洗、填充缺失值、剔除异常点并进行归一化，构建结构化的有监督学习样本。第三步，搭建包含双向 LSTM、注意力机制、Dropout 和全连接层的网络，采用 Adam 优化器与非对称损失函数，通过控制批次大小与早停策略提升泛化能力并缓解预测滞后。第四步，开展对比实验与评估：以 ARIMA 传统时序模型为基线，在测试集上比较 RMSE、MAE、 R^2 等多项指标，验证 LSTM 模型在精度与稳健性上的优势。第五步，根据最优模型输出，分析事件传播的关键节点与演变模式，探讨其在舆情早期预警、传播态势感知等场景中的落地路径，并提出引入外部特征、优化损失函数等未来研究方向。

具体研究框架如图 1 所示：

LSTM 网络热点传播速度预测模型框架

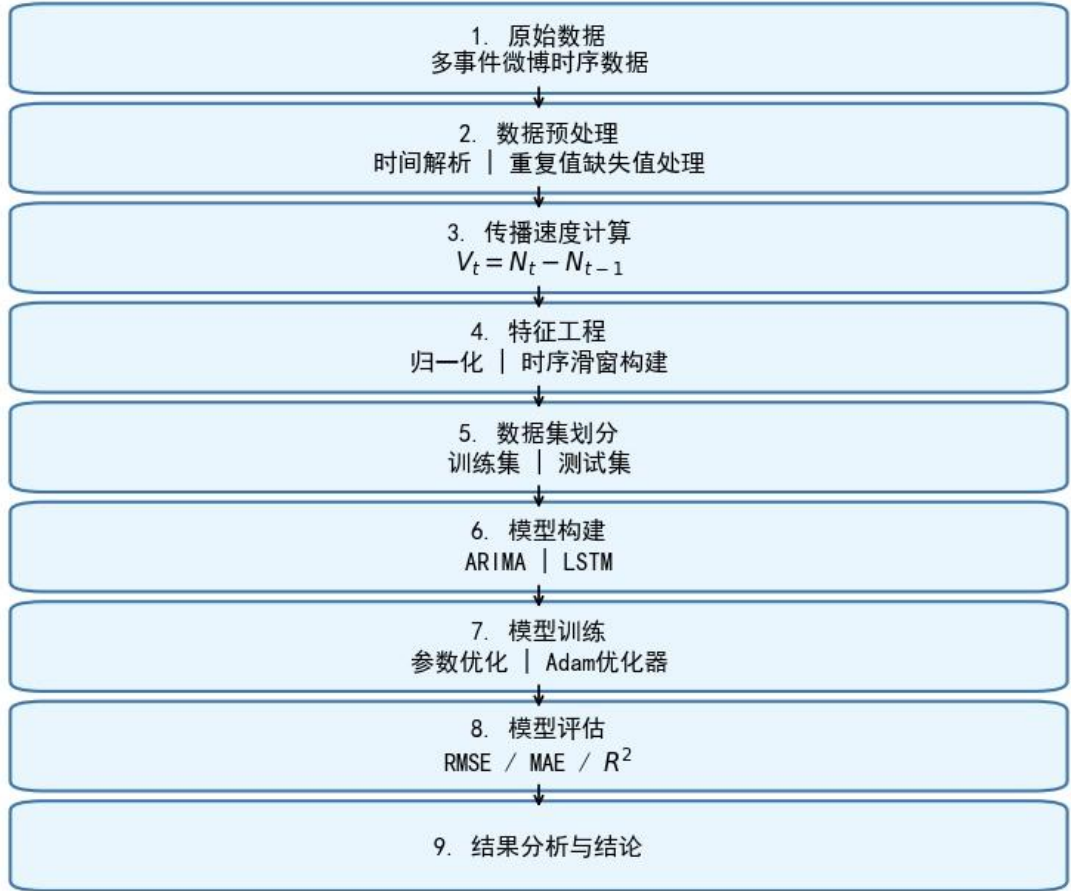


图 1 LSTM 网络热点事件传播速度预测模型框架

(二) 研究方法

本研究采用数据采集、时序构建、深度学习建模与模型优化相结合的方式开展基于 LSTM 的网络热点事件传播速度预测研究。在数据获取方面，以新浪微博为主要数据源，利用 Python 爬虫技术对热点事件相关话题进行定向抓取，获取话题热度值、评论量、点赞量、转发量等多维度原始数据，并通过数据清洗、去重、缺失值处理等步骤进行预处理，构建高质量、高时效性的热点事件时序数据集，保证了模型输入的准确性与可靠性。

在变量构建方面，以单位时间内热度值增量定义网络热点传播速度，形成连续时序变量，并对其进行 Min-Max 归一化处理，将数据映射至 $[0, 1]$ 区间，消除量纲与数值差异带来的影响，提升模型训练效率与预测的准确度。

在建模方法上，本研究选用长短期记忆神经网络（LSTM）作为核心预测模型。LSTM 能够有效处理长序列依赖问题，记忆历史传播信息，适合捕捉热点事

件在时间维度上的非线性演化规律。模型以多期历史传播数据为输入，以未来一期传播速度为输出，实现时序预测。

为了进一步提升模型性能，研究采用 Adam 优化器对模型进行优化。Adam 优化器具备自适应学习率、收敛速度快、稳定性强等优势，可以在训练过程中动态调整权重参数，有效缓解梯度消失与梯度爆炸问题，提高模型的拟合精度与泛化能力。通过不断迭代训练、误差反向传播与参数更新，最终得到收敛稳定的预测模型，实现对网络热点事件传播速度的精准预测，为舆情监测与预测提供科学方法支撑。

三、指标体系构建与数据处理

（一）指标体系构建

基础指标为原始采集数据，用于反映热点事件在社交媒体中的整体热度与用户参与程度，是构建被解释变量的基础数据来源。本研究选取微博平台公开可获取的时序指标，包括：实时热度值、累计评论量、累计点赞量、累计转发量、话题阅读量。以上指标均以固定时间间隔采集，形成连续时间序列，全面刻画事件在不同时间节点的传播规模与用户互动强度。

表 1 指标体系

变量类别	变量名称	变量符号	变量含义与说明	计算方式 / 数据来源
被解释变量	传播速度	V_t	第 t 时刻网络热点事件的传播快慢	$V_t = N_t - N_{t-1}$ ， N_t 为第 t 时刻累计阅读量
解释变量	累计阅读量	$read_{count}$	热点话题在对应时刻的总阅读量，反映事件传播广度	微博平台话题公开数据采集
解释变量	相关帖子数	$post_{count}$	与热点话题相关的发布帖子总量，反映用户参与规模	微博平台话题公开数据采集

变量类别	变量名称	变量符号	变量含义与说明	计算方式 / 数据来源
解释变量	最高点赞量	top_{like_count}	话题相关博文的点赞数量，体现用户关注度与认可程度	微博平台话题公开数据采集
解释变量	最高转发量	top_{repost_count}	话题相关博文的转发数量，反映信息扩散与传播能力	微博平台话题公开数据采集
解释变量	最高评论量	$top_{comment_count}$	话题相关博文的评论数量，体现公众讨论与互动强度	微博平台话题公开数据采集
解释变量	话题讨论量	$discuss_count$	热点话题的整体讨论规模，反映舆论发酵程度	微博平台话题公开数据采集
解释变量	互动总量	$interaction_count$	点赞、评论、转发行为的总和，综合表征用户互动水平	微博平台话题公开数据采集
解释变量	原创微博数量	$original_count$	话题相关的原创内容发布量，反映内容生产活跃度	微博平台话题公开数据采集

研究中以网络热点事件传播速度作为核心被解释变量，以时间切片下热点话题累计传播量的时序增量表示传播快慢。设定 N_t 为第 t 时刻网络热点累计热度传播量，以等时间间隔为研究步长，则第 t 时刻网络热点事件传播速度定义为：

$$V_t = N_t - N_{t-1} \quad \text{公式 (1)}$$

该指标刻画单位时间内热点新增扩散规模，数值越大代表事件传播速度越快、舆情发酵越快，作为 LSTM 模型的预测目标变量。

为了消除不同热点事件传播数据在数值量级、量纲上的差异，从而提升 LSTM 模型训练的稳定性与预测的精准度，本研究对被解释变量网络热点事件传播速度 V_t 进行 Min-Max 归一化预处理，将原始时序数据映射至统一的数值区间，从而避免因数据量级悬殊导致模型的预测偏差。即：

$$\tilde{V}_t = \frac{V_t - \min(V)}{\max(V) - \min(V)} \quad \text{公式 (2)}$$

(二) 数据采集

研究采用基于 Python 的定向采集方法，对新浪微博公开可访问的话题数据进行定时、批量获取。采集过程遵循平台规则与数据使用规范，仅获取公开信息，保证数据来源合法合规。

围绕热点事件传播速度预测这一研究目标，开展了系统、连续、规范的数据采集工作。在样本的选择上，选取了近年来在网络中传播范围较广、讨论热度较高且具有完整传播周期的典型热点事件，确保样本能够反映一般网络热点的传播规律。

为保证数据的全面性，我们采集了包括实时热度值、累计评论量、累计点赞量、累计转发量等多项能够体现传播强度与用户参与度的数据指标。所有数据均通过自动化采集工具定时获取，以固定时间间隔记录，形成连续、有序的时间序列，满足后续 LSTM 模型对时序数据的要求，如表 2 所示。

表 2 部分采集数据展示（5 条）

times tamp	pos t_cou nt	top_ like_c ount	top_ repost _count	top_ commen t_coun t	rea d_cou nt	dis cuss_ count	inter action_ count	orig inal_ count	s tat us
2026- 04- 22_01:1 0:04	992	5387	1118	806	540 84568 4	832 485	72526 02	7868 7	s ucc ess
2026- 04- 22_02:1 0:04	989	3729 0	4024	331	540 86173 4	832 494	72526 54	7868 7	s ucc ess
2026- 04- 22_03:1 0:04	979	3729 0	4024	331	540 87356 2	832 508	72526 88	7868 7	s ucc ess
2026- 04-	974	5387	1118	806	540 88501 2	832 516	72527 08	7869 1	s ucc ess

22_04:1 0:04									
2026- 04- 22_05:1 0:04	988	5387	1118	806	540 89650 6	832 526	72527 68	7869 1	s ucc ess

从整体时序特征来看，网络热点事件的传播速度呈现出高度一致演化模式，所有事件均遵循“初始由低热度短时间内快速攀升并达到峰值，然后逐步衰减至稳定低水平”的规律。事件发生初期，信息通过平台推送、用户转发与媒体报道形成快速扩散效应，传播速度在 24 - 48 小时内迅速达到峰值，体现了网络热点事件“即时发酵”的典型特征；随后，随着公众注意力的自然衰减、新热点的分流效应以及信息讨论的边际效应递减，传播速度整体呈持续下降趋势，并在 72 小时后进入长期低热度阶段，最终趋近于零。这种单峰型演化模式反映了网络热点传播的不可逆性，即热度难以在衰减过程中形成二次高峰，整体呈现一次性、集中式的传播特征。

与此同时，不同事件在传播强度、持续时长与演化节奏上存在显著差异，体现了网络热点传播的异质性特征。在峰值强度上，娱乐类、社会争议类事件的传播速度峰值普遍较高，归一化后数值多接近 1，说明此类话题更易引发公众情感共鸣并快速传播。而行业政策类、科技类事件的峰值相对较低，多集中在 0.2 - 0.4 区间，传播广度速度与参与度明显受限。在持续时长方面，多数事件的热度集中在 72 小时内消退，属于典型的“短周期热点”；但部分具有长尾效应的社会事件，传播速度在衰减过程中仍存在小幅波动，热度可持续超过一周，这类事件通常有后续信息披露、舆论反转或持续性社会影响，从而延长了传播周期。此外，不同事件的上升与衰减速率也存在分化：部分“爆发型”事件在 4 - 8 小时内即达到传播峰值，随后快速衰减；而部分话题性较弱的事件上升过程相对平缓，峰值出现时间延迟，整体演化节奏更为温和。这种差异表明，网络热点的传播速度不仅受事件本身话题性的影响，也与事件类型、信息透明度、舆论环境等多重因素密切相关，为后续 LSTM 预测模型的构建提供了数据基础与现实依据。

（三）数据预处理

我们所使用的微博热点事件原始数据包含多事件、多时序的结构化记录，在采集过程中不可避免存在重复记录、缺失观测、异常波动与无效全零样本，若直接用于建模会严重影响预测精度与稳定性，因此，我们在模型构建前对数据进行标准化预处理，主要包括重复值剔除、无效样本删除、缺失值填充与异常值修正四部分内容。

首先对原始数据中完全重复的记录进行识别与删除，避免重复样本对时序趋势与统计特征造成干扰。其次为了保证后续传播速度与热度指数计算的有效性，我们对核心热度指标均为零的无效行进行删除，具体以 $post_count$ 、 top_like_count 、 top_repost_count 、 $top_comment_count$ 四项指标为判断依据，剔除四列同时取 0 的观测样本，保留具备实际传播意义的有效数据，再次针对采集中断、平台接口异常等原因造成的数值缺失，结合数据时序特征采用线性插值法进行填充，最大限度保留时间序列的连续变化趋势，对无法插值的剩余缺失值采用 0 填充以保证数据完整可计算。最后采用 3σ 原则对数值型变量进行异常识别与修正，对超出均值 ± 3 倍标准差范围的极端值使用该列中位数进行替换，降低机器刷量、采集错误等噪声对模型训练的影响，提升数据分布的平稳性，经过上述处理，数据集实现无重复、无缺失、无异常、无无效样本，结构规范且趋势稳定，能够满足后续 LSTM 模型对时序数据的输入要求。

在去除缺失值、重复值等简单数据预处理后我们对包含多个话题指标的原始数据集实施了 Min-Max 归一化预处理。该数据集涵盖“苏超”、“五一航班大面积取消”等多个独立话题工作表，每个话题包含阅读数、点赞数、转发数、评论数、讨论数、互动数等多个维度的热度指标。这些指标数值尺度差异悬殊，例如阅读数高达数亿而点赞数仅为数万，若不处理将会导致后续分析严重偏向大数值特征。

Min-Max 归一化处理通过公式 $(x - \min) / (\max - \min)$ 对每个数值特征列进行独立线性变换。处理过程以 Python 自动化实现，遍历每个工作表的每个数值列，计算其最小值与最大值后完成映射变换，使各列数据均匀分布于 $[0, 1]$ 区间。处

理后的数据保留了原始序列的相对关系，同时消除了绝对数值差异的影响，生成了结构完整的新数据集。结果如表 3 所示。

实施归一化的核心目的是为后续 LSTM 时间序列预测模型构建标准化输入。LSTM 模型的门控机制和梯度更新对输入尺度极为敏感，未经归一化的多尺度特征会导致训练过程中梯度不平衡，引发梯度爆炸或梯度消失的问题。将特征统一至[0,1]范围，与 LSTM 中 Sigmoid 等激活函数的输出范围匹配，可确保各特征在模型中获得均衡关注，显著提升训练效率。

表 3 数据预处理后部分数据展示（5 条）

times tamp	po st_c ount	top_ like_ count	top_ repost _count	top_c omment_ count	re ad_c ount	dis cuss_ count	inter action_ count	orig inal_ count	s tat us
2026- 04- 22_01:1 0:04	0. 9708 03	0.08 9665	0.27 4588	0.759 219	0. 4916 66	0.2 22774	0.381 519	7868 7	s ucc ess
2026- 04- 22_02:1 0:04	0. 9598 54	0.74 6065	1	0.244 035	0. 4928 09	0.2 232	0.382 427	7868 7	s ucc ess
2026- 04- 22_03:1 0:04	0. 9233 58	0.74 6065	1	0.244 035	0. 4936 52	0.2 23863	0.383 021	7868 7	s ucc ess
2026- 04- 22_04:1 0:04	0. 9051 09	0.08 9665	0.27 4588	0.759 219	0. 4944 67	0.2 24242	0.383 371	7869 1	s ucc ess
2026- 04- 22_05:1 0:04	0. 9562 04	0.08 9665	0.27 4588	0.759 219	0. 4952 86	0.2 24715	0.384 419	7869 1	s ucc ess

将采集并归一化处理后得到的数据可视化处理，获得如下图表（图 2、图 3）

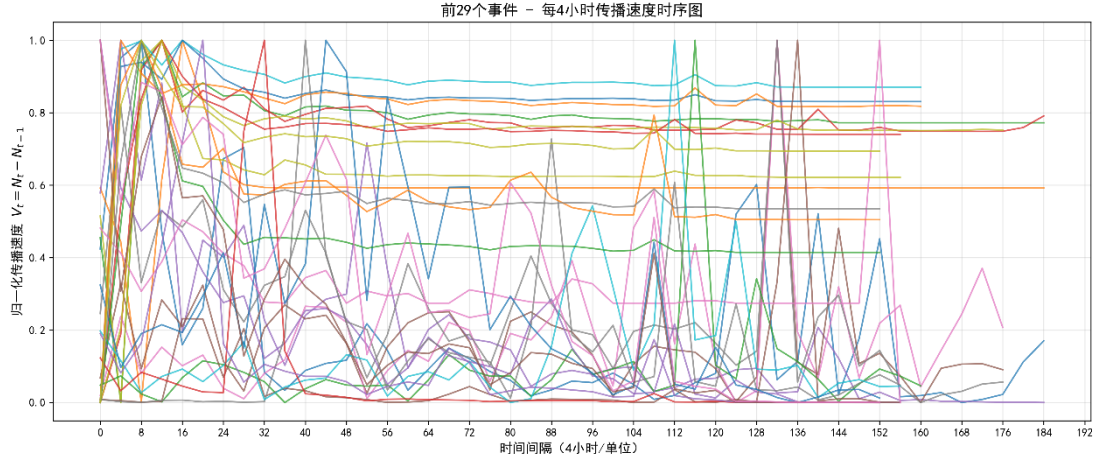


图 2 前 29 个事件每 4 小时传播速度时序图

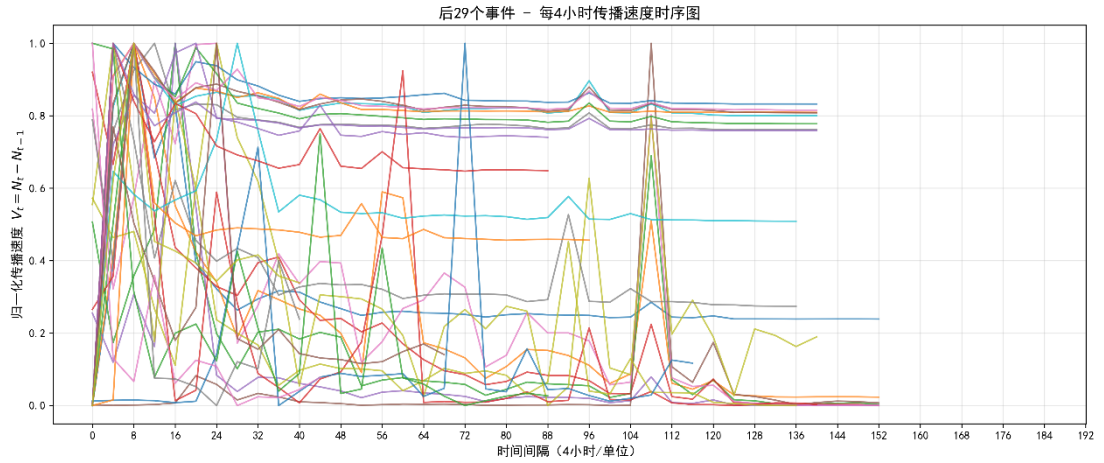


图 3 后 29 个事件每 4 小时传播速度时序图

四、基于 LSTM 的传播速度预测模型构建

(一) 预测目标与样本构造

本研究将网络热点事件的传播速度定义为单位时间内阅读量的绝对变化量，即对时间序列中相邻时刻的阅读量进行差分，再除以对应的时间间隔（以小时为单位），并取其绝对值以表征传播规模的瞬时增减强度。设第 t 个时间点的阅读量为 R_t ，与其前一时间点的时间间隔为 Δt_t ，则传播速度 s_t 计算公式为：

$$s_t = |R_t - R_{(t-1)}| / (\Delta t_t), t = 2, 3, \dots, T \quad \text{公式 (3)}$$

该指标能够有效反映信息扩散的快慢程度，且经过 Min-Max 归一化后被压缩至 $[0, 1]$ 区间，消除了量纲影响，便于模型学习。

为捕捉传播速度对历史多期信息的依赖关系，本文采用滑动窗口方法构建监督学习样本。设定窗口长度 $W=3$ ，即利用连续 3 个时间步的多维特征序列，预测紧随其后的第 4 个时间步的传播速度。每个时间步的输入特征由基础传播属性与话题标识两部分组成：

基础传播属性共 8 维，包括搜索页数据

$(post_{count}, top_{like}_{count}, top_{repost}_{count}, top_{comment}_{count})$ 和概览页数据 $(read_{count}, discuss_{count}, interaction_{count}, original_{count})$ ，均经过上述逐话题训练段归一化处理；

话题标识特征采用 one-hot 编码，向量长度等于总话题数（本数据集涉及 58 个热点话题），以二值方式标定各样本所属事件，使模型能够学习不同热点事件在传播模式上的共性与差异。

由此，每个样本的形状为 $(W, 8+K)$ ，其中 K 为话题数，完整输入维度为 $8+58=66$ 。

在样本划分上，为模拟真实预测场景中仅能使用历史数据训练、并对未来数据进行预测的需求，本文采取按话题内部时间顺序划分的训练/验证策略：对每个话题的时间序列，前 80% 的连续时段用于构建训练集，后 20% 的连续时段用于验证集。该策略避免了随机切分导致的信息泄露，更贴合舆情监测中“用过去预测未来”的实际应用逻辑。

（二）LSTM 网络结构设计

长短期记忆神经网络（LSTM）因其对序列数据中长程依赖关系的建模优势，被选为核心预测模型。本文设计的 LSTM 网络由双层 LSTM 层与三级全连接解码器组成，详细配置如下：

- 输入层：形状为 $(batch_size, 窗口长度 W, 输入特征维度 D)$ ，其中 $W=3, D=66$ 。

- 双层 LSTM 层：第一层 LSTM 接收输入序列，第二层 LSTM 在其基础上进一步提取高阶时间依赖特征。每层隐藏单元数设为 48，层间应用 Dropout 为 0.2（Pytorch 中多层 LSTM 的 dropout 仅在层间生效），以防止神经元共适应，增

强泛化性能。双层结构相比单层能够表达更复杂的时序非线性关系，同时避免过深网络在小样本条件下易出现的过拟合问题。

- 特征提取：取第 2 层 LSTM 最后一个时间步的隐藏状态作为序列的整体表征，维度为 (batch_size, 48)。

- Dropout 层：对提取的隐藏状态施加 $p=0.2$ 的 Dropout，进一步随机屏蔽部分神经元，减少模型对特定特征的依赖。

- 解码器全连接模块：采用三级全连接层逐步将高维特征压缩至标量输出，具体为：

线性层 1 (48→32) → ReLU → Dropout ($p=0.2$) → 线性层 2 (32→16) → ReLU → 线性层 3 (16→1)。

这种阶梯式降维设计既能保留足够的非线性变换能力，又能避免直接映射到单输出的过强约束，有利于稳定收敛。

模型最后输出一个标量，表示归一化后的未来一期传播速度预测值。

(三) 训练策略与优化

为使模型能够充分捕捉传播动态并在多事件数据上具有良好的泛化表现，本文采用以下训练策略：

① 损失函数

考虑到传播速度序列中可能存在个别异常波动，传统均方误差 (MSE) 对离群值敏感，易使模型偏向拟合极端值，本文选用 Huber 损失作为优化目标。Huber 损失对残差较小时表现为平方损失，较大时退化为线性损失，能够平衡拟合精度与抗野值能力，提升模型对传播速度波动的鲁棒性。

② 优化器与学习率调度

模型采用 Adam 优化器进行参数更新，初始学习率设为 0.002。Adam 算法利用梯度的一阶矩和二阶矩估计自适应调整各参数的学习率，具有收敛速度快、对超参数选择不敏感等优势，尤其适合训练中等规模的 LSTM 模型。为进一步提升收敛稳定性，引入 ReduceLROnPlateau 学习率衰减策略（监测验证损失，衰减因子 0.5，忍耐轮次 5）：当验证损失在连续 5 个 epoch 内不再下

降时，学习率自动减半。该机制可防止后期学习率过大导致损失振荡，促使模型在优化后期更精细地逼近最优解。

③早停法

为防止过拟合，设置早停忍耐轮次为 30，即当验证损失在连续 30 个 epoch 内无改善时停止训练，并恢复最佳模型参数。早停法能有效限制训练时长，同时保证模型在验证集上的泛化能力达到最优。

④批量训练与数据标准化

训练时每批次样本数为 64，训练集内样本在每轮迭代中随机打乱以增强 batch 的独立同分布性。输入特征及目标速度均已完成 Min-Max 归一化，训练过程中无需再进行额外的输入标准化。

（四）模型评估指标

为多角度评价预测性能，本文选取三个常用回归评估指标：

均方根误差（RMSE）：对较大误差赋予更高权重，反映预测值与真实值的整体偏离程度；

平均绝对误差（MAE）：直观显示预测误差的平均水平，不受误差正负方向影响；

决定系数（ R^2 ）：衡量模型对真实速度变化方差的解释比例，取值越接近 1 表明拟合优度越高。

为进一步直观验证模型的拟合效果并监测训练过程的收敛性，本研究在训练结束后加载验证集上最优模型参数，对验证样本进行预测，并绘制了两类可视化曲线。其一为预测值与真实值的对比曲线（如图 4 所示），截取验证集中连续 200 个样本点，将归一化后的实际传播速度序列与模型预测序列进行叠加展示。可以看出，预测曲线紧密跟随真实曲线的起伏趋势，尤其在传播速度的局部峰值与谷值处均保持了较高的一致性，表明 LSTM 模型能够有效提取序列中的动态特征，对速度演化趋势具备良好的捕获能力。

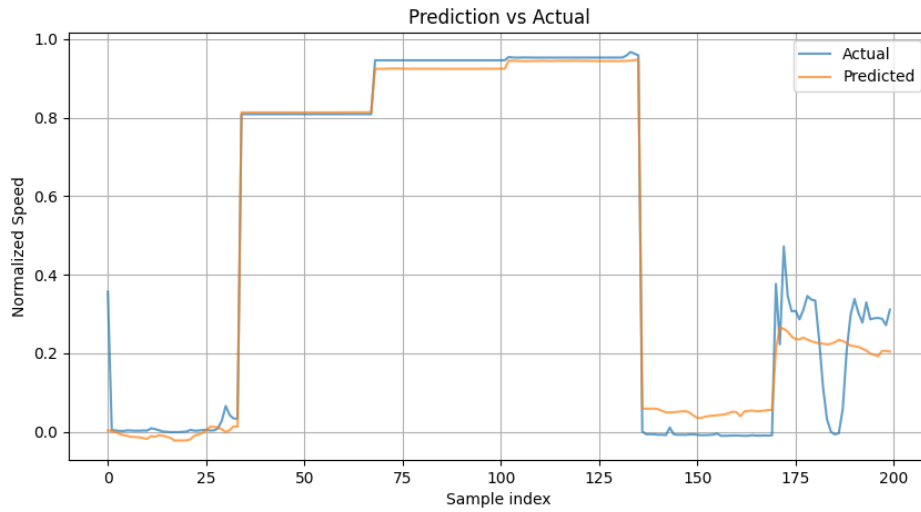


图 4 预测值与真实值的对比曲线

其二为训练损失与验证损失随训练轮次（Epoch）的变化曲线（如图 5 所示）。图中同时记录每个 Epoch 下的训练集 MSE 损失和验证集 MSE 损失。观察可知，两类损失均在初始数轮内快速下降，随后逐渐减缓并趋于平稳，且验证损失未出现明显抬升，说明早停机制（Early Stopping）在防止过拟合方面起到了积极作用，模型泛化性能得到有效保障。

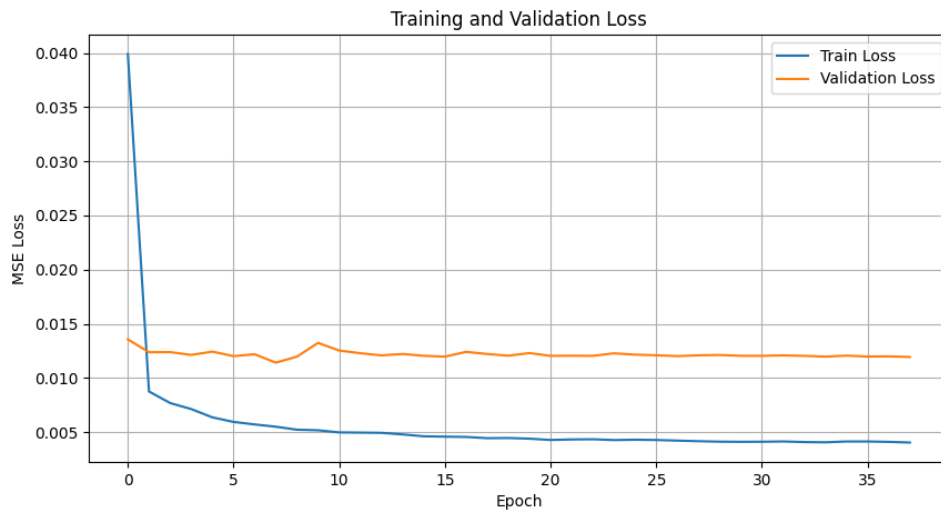


图 5 训练损失与验证损失随训练轮次（Epoch）的变化曲线

五、实证数据结果检验与模型对比

为系统评估模型在网络热点传播速度预测任务上的性能，本研究采用均方根误差（RMSE）、平均绝对误差（MAE）及决定系数（ R^2 ）作为核心评价指标，

对传统 ARIMA 模型与所构建的双层 LSTM 模型进行对比分析。其中，LSTM 模型在训练过程中通过早停机制（Early Stopping）控制过拟合，最终于第 38 轮训练（epoch）停止，其在验证集上取得的最佳结果为： $RMSE = 0.2439$ ， $MAE = 0.0453$ ， $R^2 = 0.7463$ 。上述结果表明，在归一化尺度下，该模型能够较为准确地捕捉传播速度的演化趋势——平均绝对误差仅约 0.045，相当于归一化区间的 4.5%，且对速度波动的解释程度达到约 74.6%，初步验证了 LSTM 方法用于该任务的有效性。

进一步地，将训练好的双层 LSTM 模型与传统 ARIMA 模型在相同测试集上进行对比，两类模型在捕捉传播过程动态规律上的性能差异如下图（图 4）：

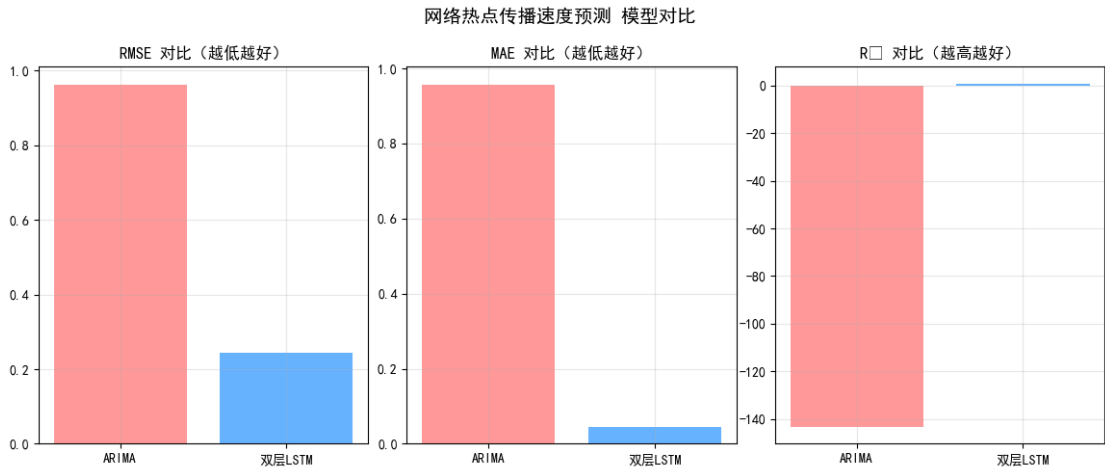


图 6 网络热点事件传播速度预测模型对比

实验结果表明，ARIMA 模型的 RMSE 为 0.9626，MAE 为 0.9567，两项误差指标均处于较高水平，说明模型预测值与真实值偏差显著，难以反映传播速度的真实变化趋势。更关键的是其 R^2 值为 -143.24，呈现出严重的负值，这意味着该模型的预测完全无法解释网络热点传播过程中的非线性波动与动态突变特征。

而相比之下，双层 LSTM 模型展现出压倒性的性能优势，实现了误差控制与拟合能力的双重突破。在误差指标上，双层 LSTM 的 RMSE 仅为 0.2439，较 ARIMA 模型降低约 74.7%；MAE 仅为 0.0453，较 ARIMA 模型降低约 95.3%，两项误差指标均实现断崖式下降，说明模型预测值与真实值的偏差被大幅压缩，预测精度得到质的提升。在拟合程度方面，双层 LSTM 的 R^2 值达到

0.7463，表明该双层 LSTM 模型能够解释约 74.6% 的数据波动，对网络热点传播过程中的时序依赖关系、非线性变化规律以及突发式增长特征具备较强的捕捉能力。

ARIMA 作为线性时间序列模型，仅能捕捉数据的线性平稳变化规律，无法适配网络热点传播过程中受用户互动、舆论发酵等多重因素驱动的非线性、动态突变特性；而双层 LSTM 凭借其独特的门控循环结构，能够有效建模长序列中的长期依赖关系，自动学习复杂时序数据的非线性特征，较好地适配网络热点传播的动态演化规律。综上，双层 LSTM 模型在预测精度、拟合能力与场景适配性上均显著优于传统 ARIMA 模型。

六、研究结论

我们围绕网络热点事件传播速度精准预测问题，构建并验证了基于 LSTM 的深度学习预测模型，以新浪微博多话题、多周期的时序数据为样本，完成从指标构建、数据预处理、模型训练到多方法对比的全流程实证分析，得到如下研究结论：

第一，网络热点事件传播速度具有典型的非线性、非平稳时序特征，整体呈现“短时快速攀升至峰值、逐步衰减趋稳”的单峰演化规律，不同类型事件在传播峰值、上升速率、持续周期上存在显著差异，社会民生类事件传播强度更高、爆发更快，科技政策类事件传播节奏更平缓，传统线性时序模型难以有效拟合此类复杂动态变化。

第二，科学规范的数据预处理能够显著提升预测效果，通过剔除重复记录、删除 post_count、top_like_count、top_repost_count、top_comment_count 全零无效样本、线性插值补全缺失值、 3σ 原则修正异常值，并结合 Min-Max 归一化消除量纲差异，可有效降低噪声干扰，使数据更贴合 LSTM 模型的输入要求，为精准预测提供可靠基础。

第三，本研究设计的双层 LSTM 模型在传播速度预测任务中表现稳定、性能优异。模型以窗口长度 3、输入维度 66（含 8 维传播特征与 58 个话题 One-Hot 编码）为输入结构，采用双层 LSTM 搭配三级全连接层，使用 Huber

损失、Adam 优化器与早停策略进行训练，在验证集上实现 $RMSE=0.2439$ 、 $MAE=0.0453$ 、 $R^2=0.7463$ ，能够较好捕捉传播序列的长期依赖与波动突变特征。

第四，LSTM 模型显著优于传统 ARIMA 模型。对比实验显示，ARIMA 模型因无法拟合非线性波动导致预测失效， R^2 为负值、误差指标居高不下；而 LSTM 模型在误差控制与解释能力上实现大幅提升，RMSE 相对降低约 74.7%，MAE 相对降低约 95.3%，充分证明深度学习方法在舆情时序预测上的适用性与优越性。

总体来看，基于 LSTM 的网络热点事件传播速度预测模型能够有效实现对热点演化节奏的精准预判，可为舆情早期预警、趋势研判和引导处置提供量化支撑。研究在视角上从热度预测转向速度预测、在指标上构建多维度时序输入、在方法上验证 LSTM 适配性，既拓展了舆情量化分析的理论边界，也具备较强的现实应用价值。

七、现实意义与价值

本研究提出的基于 LSTM 的网络热点事件传播速度预测模型具有多层次的现实意义与应用价值。

在理论层面，本研究突破了传统舆情分析聚焦静态热度值的局限，将视角延伸至动态的“传播速度”这一核心指标。实证结果不仅证实了 LSTM 在捕捉网络热点传播中非线性、长程依赖时序特征上的有效性，也为后续探索网络信息扩散的内在机制提供了新的工具与理论视角。

在公共治理与实践应用层面，本模型能够为舆情监测与应急管理提供关键决策支持。通过对传播速度的精准预测，有关部门可提前研判舆情扩散的高峰与加速节点，将管理模式转向事前的主动预警和干预。例如，在突发公共事件中，依托模型的预测结果可抢先发布权威信息、精准调配引导资源，有效遏制谣言传播，提升响应时效与治理效能。同时，该模型也可为社交媒体平台的内容审核、流量分配及风险防控提供科学依据，辅助平台实施差异化管理策略，在保障信息传播效率的同时维护网络生态秩序。

在行业与风险管理层面，本模型为企业和机构的品牌与声誉管理提供了前瞻性工具。通过早期识别并评估话题的扩散与传播风险，有助于相关部门及主

体及时制定应对策略，减少舆情危机带来的潜在损失，提升组织的风险抵御能力。

综上所述，本研究不仅在模型构建与应用上创新，提升了舆情预测的精度，更将前沿深度学习技术与网络空间治理的现实需求紧密结合，为构建数据驱动、智能敏捷的网络综合治理体系提供了具有可操作性的技术方案和决策支持。

八、政策建议

结合网络热点事件传播速度预测模型的研究结论与实证分析结果，立足社交媒体舆情治理、公共信息管理与应急响应现实需求，我们从舆情监测、风险预警、平台治理、公共治理四个层面提出以下政策建议：

第一，构建基于时序预测的热点事件常态化监测与预警机制。政府舆情管理部门可以将 LSTM 传播速度预测模型纳入日常舆情监测体系，依托微博、抖音等平台实时热度数据，对热点事件的爆发时点、传播峰值与持续时长进行动态预测，实现从“事后处置”向“事前预判”转型，针对公共安全、民生服务类事件，可设置分级预警阈值，当预测传播速度达到临界值时及时启动响应，降低负面舆情扩散风险。

第二，完善社交媒体平台热点内容引导与传播干预机制。平台方应根据传播速度预测结果，对不同类型热点事件采取差异化管理策略：对政策宣传、正能量社会事件，可借助预测结果优化流量分配与推送时机，扩大正面舆论影响力，而对于突发性、高风险事件，在预测到快速扩散趋势时应及时加强内容审核与风险提示，规范传播秩序，维护网络生态稳定。

第三，强化政府部门间舆情信息共享与协同处置机制。建议建立跨部门、跨平台的舆情数据共享通道，统一热点事件热度指标、传播速度等关键定义，破除数据壁垒。宣传、网信、应急等部门可依据预测结果协同制定应对方案，提升热点事件响应的时效性与精准性，形成“监测-预测-处置-反馈”的闭环治理模式。

第四，推进基层治理与舆情服务的精细化与人性化。对公共服务类热点事件，相关部门可依托传播速度与热度趋势判断公众关切焦点，主动发布权威信

息、回应社会诉求，减少谣言滋生空间，同时，在舆情干预中坚持依法依规、审慎适度，兼顾传播管控与公众知情权、表达权，提升政府公信力与治理亲和力。

九、研究局限与未来展望

（一）研究局限

第一，数据来源与覆盖范围存在局限：我们主要采用微博平台公开热点时序数据，样本在事件类型、地域分布、时间跨度上仍存在一定局限性，未纳入抖音、知乎、微信等多平台数据，模型在跨平台泛化能力上有待进一步验证。

第二，影响因素考虑相对单一：模型仅以转发、评论、点赞等时序热度指标作为输入，未将用户画像、情感倾向、网络结构、媒体介入、政策事件等外部影响因素纳入建模体系，对传播速度背后的复杂动因解释不够充分。

第三，模型结构与参数仍有优化空间：我们采用基础 LSTM 结构完成预测任务，未与 Transformer、GRU、CNN-LSTM 等组合模型进行深度对比，在长时序依赖捕捉、小样本预测稳定性方面仍存在提升空间。

第四，应用场景的落地性有待加强：本研究侧重于预测方法验证与效果分析，尚未形成可直接部署的轻量化、可视化系统工具，在实时接口、dashboard 展示、预警推送等工程化应用方面仍需完善。

（二）未来展望

未来可从数据、模型、机制、应用四个方向进一步拓展深化。我们后续可以拓展多平台、多模态数据来源，融合文本情感、话题关联、社会关系网络等信息，构建更全面的特征体系，并且引入混合深度学习模型与集成学习方法，结合注意力机制、贝叶斯优化等技术提升预测精度与鲁棒性，在此基础上深化机理研究，探索不同类型热点事件的传播规律差异，形成分场景、可解释的舆情预测理论框架，最后希望可以推动研究成果向实用化系统转化，开发可视化预警平台，为政府、平台与基层治理提供可直接使用的工具支持，不断提升网络空间治理体系和治理能力现代化水平。

参考文献

- [1] 郑忠. 基于长短时记忆模型的热点事件计算机智能预测方法[C]// 未注明会议论文集. 2021: 795-797.
- [2] 潘国勇, 任晔, 俞海英, 等. 基于 LSTM 神经网络的四川泸定 M6.8 级地震后公众舆情情感演化建模与可视化[J]. 组合数学与组合计算, 2025, 127b: 9515-9535.
- [3] 姜雨川, 贾朝龙, 秦云义, 等. 社交网络中主题传播分类的多模态框架[J]. 未注明, 2023.
- [4] 程唐. 基于时序数据建模技术的社交媒体舆论动态预测研究[J]. 国际住房科学及其应用杂志, 2025, 46(3): 6554-6565.
- [5] 鲁琰珂. 基于 LDA 与 GWO-LSTM 的社会热点事件网络舆情主题热度预测研究[D]. 武汉: 中南民族大学, 2023.
- [6] 庞敏. 基于 LSTM 混合模型的时间序列预测[D]. 武汉: 华中科技大学, 2019.
- [7] 魏德宾, 范中林, 杨力. 基于 VMD 与改进 LSTM 模型的网络流量预测[J]. 计算机仿真, 2025, 42(8): 524-529.
- [8] 李若晨, 肖人彬. 基于改进狼群算法优化 LSTM 网络的舆情演化预测[J]. 复杂系统与复杂性科学, 2024, 21(1): 1-11.
- [9] 王柯萱, 高焯, 李松. 基于情感分类和库普曼算子的网络舆情热度预测[J]. 数字图书馆论坛, 2025, 21(8): 11-22.
- [10] 马永军, 郎威. 基于融合注意力机制 LSTM 的网络舆情趋势预测[J]. 计算机仿真, 2023, 40(1): 493-498.
- [11] 刘定一, 沈阳阳, 詹天明, 等. 融合微博热点分析和 LSTM 模型的网络舆情预测方法[J]. 江苏大学学报(自然科学版), 2021, 42(5): 546-553.
- [12] 安璐, 惠秋悦. 热点事件情境下微博舆情反转预测[J]. 信息资源管理学报, 2022, 12(3): 21-34.
- [13] 曾庆康. 网络热点话题传播模型与预测研究[D]. 北京: 北京邮电大学, 2020.
- [14] 吕晶. 网络热点话题的传播模型分析[D]. 北京: 北京邮电大学, 2013.
- [15] 鲁宁. 基于事件传播网络的热点事件爆发预测研究[D]. 北京: 北京工业大学, 2020.
- [16] 李彪. 网络事件传播空间结构及其特征研究——以近年来 40 个网络热点事件为例[J]. 新闻与传播研究, 2011(3): 90-96+112.
- [17] MU Guangyu, LIAO Zehan, LI Jiaxue, et al. IPSO-LSTM hybrid model for predicting online public opinion trends in emergencies[J]. *PLOS ONE*, 2023, 18(10): e0292677.
- [18] ZHAO Jinghua, HE Huihong, ZHAO Xiaohua, et al. Modeling and simulation of microblog-based public health emergency-associated public opinion communication[J]. *Information Processing and Management*, 2022, 59: 102846.
- [19] WANG Jintao, YIN Yulong, WEI Lina. Modeling public opinion dynamics in social networks using a GAN-SEIR framework[J/OL].

- Social Network Analysis and Mining*, 2025, 15: 40[2025-04-16].
<https://doi.org/10.1007/s13278-025-01426-x>
- [20] 沈旺, 李欣, 孙珂, 等. 网络舆情事件用户在线评论行为预测研究[J].
现代情报, 2025, 45(12): 118-129.

附录

附件 1: 微博平台数据抓取代码 (部分核心代码)

```
# 基于近期热点主题/热搜榜单进一步筛选出特定词条, 对其搜索+概览页面相关数据进行抓取 (省略了较为繁杂的 excel 追加和词条列表更新逻辑, 仅展示核心抓取函数及依赖库)
# 为了规避反爬机制, 选择错开时间执行两大类数据的抓取, 故分为两个脚本。
import requests
from datetime import datetime
import time
from fake_useragent import UserAgent
import random
from urllib.parse import quote
import logging

#初始化配置 (主要用于搜索页)
HEADERS = {
    'Referer': 'https://m.weibo.cn/',
    'User-Agent': UserAgent().random,
    'Accept': 'application/json, text/plain, */*',
    'Accept-Language': 'zh-CN,zh;q=0.9,en;q=0.8', # 移动版 API 需要加入更多参数以提高抓取成功率
    'X-Requested-With': 'XMLHttpRequest',
    'MWeibo-Pwa': '1',
}
COOKIES = {
    'SUB': 'COOKIE_SUB', # 实际需要切换成用户登录状态下的 cookies
}
KEYWORDS = ['default_keyword'] # 实际会不断更新

# 两大类数据分别使用两个核心函数封装抓取逻辑
def get_search_result_data(keyword):
    """通过移动版 API 获取关键词搜索结果总数"""
    url = 'https://m.weibo.cn/api/container/getIndex'
    params = {
        'containerid': f'100103type=1&q={keyword}',
        'page_type': 'searchall'
    }
    try:
        # 随机延时, 降低反爬风险
        time.sleep(random.uniform(0.5, 1.5))
        response = requests.get(url, headers=HEADERS, cookies=COOKIES,
                                params=params, timeout=10)
```

```

        if response.status_code != 200:
            logging.warning(f"请求失败, 状态码: {response.status_code}, 关键词: {keyword}")
            return {'post_count': None, 'top_like_count': None}

        # 尝试解析 JSON
        data = response.json()
        total = data.get('data', {}).get('cardlistInfo', {}).get('total')

        # 初始化聚合数据
        top_like = 0
        top_repost = 0
        top_comment = 0

        cards = data.get('data', {}).get('cards', [])
        for card in cards:
            if card.get('card_type') == 9: # 微博卡片类型
                mblog = card.get('mblog', {})
                # 提取单条微博数据
                like = mblog.get('attitudes_count') or 0
                repost = mblog.get('reposts_count') or 0
                comment = mblog.get('comments_count') or 0
                if like > top_like:
                    top_like = like
                if repost > top_repost:
                    top_repost = repost
                if comment > top_comment:
                    top_comment = comment

        return {
            'post_count': int(total) if total else 0,
            'top_like_count': top_like,
            'top_repost_count': top_repost,
            'top_comment_count': top_comment,
        }

    except requests.exceptions.JSONDecodeError:
        logging.error(f"JSON 解析失败, 关键词: {keyword}, 响应内容开头: {response.text[:200]}")
        return {
            'post_count': None,
            'top_like_count': None,
            'top_repost_count': None,

```

```

        'top_comment_count': None,
    }
except Exception as e:
    logging.error(f"其他异常: {e}")
    return {
        'post_count': None,
        'top_like_count': None,
        'top_repost_count': None,
        'top_comment_count': None,
    }

def get_topic_overview(keyword):
    """获取话题的整体概览数据（阅读量、讨论量等）"""
    # 对关键词进行 URL 编码，以处理空格和特殊字符
    encoded_keyword = quote(keyword)
    url =
f'https://m.s.weibo.com/ajax_topic/detail?q={encoded_keyword}&show_rank_info=
1'
    # 概览页与搜索页请求头和参数详细程度要求不同。为了便于阅读维护，这里直接
    重写 headers
    headers = {
        'User-Agent': UserAgent().random,
        'Referer': 'https://m.s.weibo.com/',
        'Accept': 'application/json, text/plain, */*',
        'X-Requested-With': 'XMLHttpRequest',
    }
    # COOKIES 使用全局即可
    cookies = COOKIES
    try:
        time.sleep(random.uniform(0.5, 1.5))
        response = requests.get(url, headers=headers, cookies=cookies,
timeout=10)
        if response.status_code != 200:
            logging.warning(f"获取话题概览失败，关键词: {keyword}，状态码:
{response.status_code}")
            return None
        data = response.json()
        if data.get('msg') == 'success':
            count_info = data.get('data', {}).get('baseInfo',
{}).get('count', {})
            return {
                'read_count': count_info.get('read'),
                'discuss_count': count_info.get('mention'), # 注意：使用
'mention'

```

```

        'interaction_count': count_info.get('interact'),
        'original_count': count_info.get('ori_m'), # 原创微博量
    }
    else:
        logging.warning(f"接口返回失败, 关键词: {keyword}, 消息: {data.get('msg')}")
        return None
    except Exception as e:
        logging.error(f"获取话题概览时异常, 关键词: {keyword}, 错误: {e}")
        return None

def track_keywords():
    """核心追踪任务, 包括列表更新及两大类数据整合 (这里仅展示数据组合逻辑)"""
    logging.info(f"当前追踪关键词 ({len(KEYWORDS)}个): {KEYWORDS}")
    results = []
    for kw in KEYWORDS:
        # 获取搜索页面数据
        detail_data = get_search_result_data(kw)
        # 获取概览页面数据
        overview_data = get_topic_overview(kw)
        timestamp = datetime.now().strftime('%Y-%m-%d_%H:%M:%S') # 单次获取词
        # 条数据时获取时间戳, 消除应对反爬随机延时带来的时间差

        # 组合数据
        record = {
            'timestamp': timestamp,
            'keyword': kw,
            # 明细数据
            'post_count': detail_data.get('post_count'),
            'top_like_count': detail_data.get('top_like_count'),
            'top_repost_count': detail_data.get('top_repost_count'),
            'top_comment_count': detail_data.get('top_comment_count'),
            # 概览数据(如前面加类别这里也要加)
            'read_count': overview_data.get('read_count') if overview_data
        else None,
            'discuss_count': overview_data.get('discuss_count') if
        overview_data else None,
            'interaction_count': overview_data.get('interaction_count') if
        overview_data else None,
            'original_count': overview_data.get('original_count') if
        overview_data else None,
            'status': 'success' if detail_data.get('post_count') is not None
        else 'fail'

```

```

    }
    results.append(record)
    logging.info(f" 第 {len(results)} 条关键词 '{kw}' -> 数据合集:
{record}") # 在控制台展示整合后的数据，便于调试

```

附件 2: 数据 min-max 归一化处理代码（部分核心代码）

```

target_columns = ['post_count', 'top_like_count', 'top_repost_count',
                  'top_comment_count', 'read_count', 'discuss_count',
                  'interaction_count']

print(f"找到 {len(sheet_names)} 个 Sheet: {sheet_names}")

for sheet_name in sheet_names:
    print(f"\n 处理 Sheet: {sheet_name}")

    # 读取单个 Sheet
    df = pd.read_excel(xls, sheet_name=sheet_name)
    print(f" 原始数据形状: {df.shape}")

    # 确保只对数值列进行处理
    numeric_cols = [col for col in target_columns if col in df.columns]
    print(f" 将归一化的列: {numeric_cols}")

    if numeric_cols:
        # 执行 Min-Max 归一化
        df_normalized = min_max_normalize(df, numeric_cols)

        # 保存结果
        results[sheet_name] = df_normalized

        # 显示结果
        print(f" 归一化完成:")
        print(df_normalized[numeric_cols])
    else:
        # 如果没有需要处理的列，保留原始数据
        results[sheet_name] = df
        print(f" 无需要归一化的列，保留原始数据")

```

附件 3: LSTM 模型构建与训练代码（部分核心代码）

```

# ===== 配置参数（便于整体直接调节） =====
DATA_PATH = "normalized_topic.xlsx" # 数据文件路径
WINDOW_SIZE = 3 # 滑动窗口大小（可调超参数）

```

```

HIDDEN_SIZE = 48                                # LSTM 隐藏层大小
NUM_LAYERS = 2                                    # LSTM 层数（双层以增强非线性表达能力）
DROPOUT = 0.2                                     # Dropout 比例
BATCH_SIZE = 64
EPOCHS = 100
LEARNING_RATE = 0.002
PATIENCE = 30                                     # 早停 patience
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# ===== 0. 工具函数 =====
def parse_timestamp(ts_str):
    """将字符串时间戳转换为 datetime 对象"""
    return datetime.strptime(ts_str, "%Y-%m-%d_%H:%M:%S")

def compute_speed(series, time_deltas):
    """
    计算传播速度：单位时间 read_count 的变化量
    输入：series (pandas Series, read_count 值), time_deltas (小时为单位的时间间隔)
    返回：传播速度数列，第一个时刻为 NaN
    """
    speed = (series.diff()) / time_deltas # 差商
    speed = speed.abs()
    speed.iloc[0] = np.nan # 第一个时刻无差商
    return speed

# 固定随机种子以减少随机性引发的显著实验差异
def set_seed(seed=42):
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

# ===== 3. 模型定义 =====
class ResearchLSTM(nn.Module):
    """
    LSTM 预测模型
    结构：双层 LSTM -> Dropout -> FC1(隐藏单元→32) -> ReLU -> Dropout ->
    FC2(32→16) -> ReLU -> FC3(16→1)
    描述：
        双层 LSTM 能捕捉更复杂的时间依赖，同时避免过深网络导致的过拟合和训练困难。
    """

```

Dropout 在 LSTM 输出后使用，减少神经元共适应，增强泛化能力。

三组全连接层逐步压缩维度，模仿解码器结构，将高维特征映射到标量输出，增加非线性表达能力。

```
"""
def __init__(self, input_size, hidden_size, num_layers, dropout):
    super(ResearchLSTM, self).__init__()
    self.lstm = nn.LSTM(input_size=input_size,
                        hidden_size=hidden_size,
                        num_layers=num_layers,
                        batch_first=True,
                        dropout=dropout if num_layers > 1 else 0) # 仅多
层时内部 dropout 有效
    self.dropout = nn.Dropout(dropout)
    self.fc1 = nn.Linear(hidden_size, 32)
    self.relu1 = nn.ReLU()
    self.fc2 = nn.Linear(32, 16)
    self.relu2 = nn.ReLU()
    self.fc3 = nn.Linear(16, 1)

    def forward(self, x):
        # x: (batch, seq_len, input_size)
        lstm_out, (h_n, c_n) = self.lstm(x) # lstm_out: (batch,
seq_len, hidden_size)
        last_hidden = lstm_out[:, -1, :] # (batch, hidden_size)
        out = self.dropout(last_hidden)
        out = self.fc1(out)
        out = self.relu1(out)
        out = self.dropout(out)
        out = self.fc2(out)
        out = self.relu2(out)
        out = self.fc3(out) # (batch, 1)
        return out

# ===== 4. 训练与验证 =====
def train_model(model, train_loader, val_loader, epochs, learning_rate,
patience):
    criterion = nn.HuberLoss() # 若采用 MSELoss, 实测模型表现会略差
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer,
mode='min', factor=0.5, patience=5)

    best_val_loss = float('inf')
    patience_counter = 0
    train_losses, val_losses = [], []
```

```

for epoch in range(epochs):
    # 训练阶段
    model.train()
    train_loss = 0.0
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs, y_batch)
        loss.backward()
        optimizer.step()
        train_loss += loss.item() * X_batch.size(0)
    train_loss /= len(train_loader.dataset)
    train_losses.append(train_loss)

    # 验证阶段
    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            X_batch, y_batch = X_batch.to(DEVICE), y_batch.to(DEVICE)
            outputs = model(X_batch)
            loss = criterion(outputs, y_batch)
            val_loss += loss.item() * X_batch.size(0)
    val_loss /= len(val_loader.dataset)
    val_losses.append(val_loss)

    # 学习率调度
    scheduler.step(val_loss)
    print(f"Epoch {epoch+1}/{epochs}, Train Loss: {train_loss:.6f}, Val
Loss: {val_loss:.6f}")

    # 早停检查
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        patience_counter = 0
        torch.save(model.state_dict(), "best_model.pth")
    else:
        patience_counter += 1
        if patience_counter >= patience:
            print(f"\nEarly stopping triggered after {epoch+1} epochs.")
            break
    return train_losses, val_losses

```

```

# ===== 5. 性能评估 =====
def evaluate_model(model, data_loader):
    model.eval()
    predictions, actuals = [], []
    with torch.no_grad():
        for X_batch, y_batch in data_loader:
            X_batch = X_batch.to(DEVICE)
            pred = model(X_batch).cpu().numpy()
            predictions.extend(pred)
            actuals.extend(y_batch.numpy())
    predictions = np.array(predictions).flatten()
    actuals = np.array(actuals).flatten()

    rmse = np.sqrt(np.mean((predictions - actuals) ** 2))
    mae = np.mean(np.abs(predictions - actuals))
    ss_res = np.sum((actuals - predictions) ** 2)
    ss_tot = np.sum((actuals - np.mean(actuals)) ** 2)
    r2 = 1 - ss_res / ss_tot if ss_tot != 0 else 0.0

    print(f"————— 验证集评估 —————")
    print(f"RMSE: {rmse:.6f}")
    print(f"MAE : {mae:.6f}")
    print(f"R²   : {r2:.6f}")
    return predictions, actuals

def plot_predictions(actuals, predictions, title="Prediction vs Actual"):
    plt.figure(figsize=(10, 5))
    plt.plot(actuals[:200], label="Actual", alpha=0.7)
    plt.plot(predictions[:200], label="Predicted", alpha=0.7)
    plt.title(title)
    plt.xlabel("Sample index")
    plt.ylabel("Normalized Speed")
    plt.legend()
    plt.grid(True)
    plt.show()

# ===== 主流程 =====
if __name__ == "__main__":
    set_seed(42) # 固定随机种子以保证实验可复现。
    # 1. 加载并构建数据集
    print("正在加载数据...")
    X_train, y_train, X_val, y_val, input_size = load_all_sheets(DATA_PATH,
train_ratio=0.8)

```

```

print(f"训练样本数: {len(X_train)}, 验证样本数: {len(X_val)}")
print(f"输入特征维度: {input_size}, 窗口长度: {WINDOW_SIZE}")

# 2. 创建 DataLoader
train_dataset = HotEventDataset(X_train, y_train)
val_dataset = HotEventDataset(X_val, y_val)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE,
shuffle=False)

# 3. 初始化模型
model = ResearchLSTM(input_size=input_size, hidden_size=HIDDEN_SIZE,
num_layers=NUM_LAYERS, dropout=DROPOUT).to(DEVICE)
print(model)

# 4. 训练
print("开始训练...")
train_losses, val_losses = train_model(model, train_loader, val_loader,
epochs=EPOCHS,
learning_rate=LEARNING_RATE,
patience=PATIENCE)

# 5. 加载最佳模型并评估
model.load_state_dict(torch.load("best_model.pth"))
predictions, actuals = evaluate_model(model, val_loader)
plot_predictions(actuals, predictions)

# 6. 绘制损失曲线
plt.figure(figsize=(10, 5))
plt.plot(train_losses, label='Train Loss')
plt.plot(val_losses, label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('MSE Loss')
plt.title('Training and Validation Loss')
plt.legend()
plt.grid(True)
plt.show()

```